



Round 3 Quests

📅 Calendar	👤 Round.3
🏷 Tag	BE_L2 round3 task



Round 3 Quests



Implementation Quest

- * 도메인 모델링을 통해 Product, Brand, Like, Order 기능의 핵심 개념을 **Entity, Value Object, Domain Service** 등 적합한 코드로 작성합니다.
- * 레이어드 아키텍처 + DIP 를 적용해 유연하고 테스트 가능한 구조로 구성합니다.
- * **Application Layer**를 경량 수준으로 구성하여, 도메인 객체를 조합하는 흐름을 실제 구현해봅니다.
- * 단위 테스트를 작성하여 도메인 로직의 정합성과 규칙을 검증합니다.



Claude 가 준수해야할 Rule 을 확장하기

- **CLAUDE.md** 에 객체지향, 도메인 모델링 규칙 섹션을 추가해 봅니다.
- **CLUADE.md** 에 아키텍처 전략, 패키지 구성 방향 등을 추가해 봅니다.
 - 작성 예시

도메인 & 객체 설계 전략

- 도메인 객체는 비즈니스 규칙을 캡슐화해야 합니다.
- 애플리케이션 서비스는 서로 다른 도메인을 조립해, 도메인 로직을 조정하여 기능을 제공해야 합니다.
- 규칙이 여러 서비스에 나타나면 도메인 객체에 속할 가능성이 높습니다.
- 각 기능에 대한 책임과 결합도에 대해 개발자의 의도를 확인하고 개발을 진행합니다.

아키텍처, 패키지 구성 전략

- 본 프로젝트는 레이어드 아키텍처를 따르며, DIP (의존성 역전 원칙) 을 준수합니다.
- API request, response DTO와 응용 레이어의 DTO는 분리해 작성하도록 합니다.
- 패키징 전략은 4개 레이어 패키지를 두고, 하위에 도메인 별로 패키징하는 형태로 작성합니다.
 - 예시
 - > /interfaces/api (presentation 레이어 - API)
 - /application/.. (application 레이어 - 도메인 레이어를 조합해 사용 가능한 기능을 제공)
 - /domain/.. (domain 레이어 - 도메인 객체 및 엔티티, Repository 인터페이스가 위치)
 - /infrastructure/.. (infrastructure 레이어 - JPA, Redis 등을 활용해 Repository 구현체를 제공)



구현 과제

- 상품, 브랜드, 좋아요, 주문 기능의 **도메인 모델 및 도메인 서비스**를 구현합니다.
- 도메인 간 협력 흐름을 설계하고, 필요한 로직을 **도메인 서비스**로 분리합니다.
- Application Layer에서 도메인 객체를 조합하는 흐름을 구현합니다.
(예: `ProductFacade.getProductDetail(productId)` → `Product + Brand + Like 조합`)
- Repository Interface 와 구현체는 분리하고, 테스트 가능성을 고려한 구조를 설계합니다.
- 모든 핵심 도메인 로직에 대해 단위 테스트를 작성하고, 예외/경계 케이스도 포함합니다.



Checklist

Product / Brand 도메인

- ☐ 상품 정보 객체는 브랜드 정보, 좋아요 수를 포함한다.
- ☐ 상품의 정렬 조건(`latest` , `price_asc` , `likes_desc`) 을 고려한 조회 기능을 설계했다
- ☐ 상품은 재고를 가지고 있고, 주문 시 차감할 수 있어야 한다
- ☐ 재고의 음수 방지 처리는 도메인 레벨에서 처리된다

Like 도메인

- ☐ 좋아요는 유저와 상품 간의 관계로 별도 도메인으로 분리했다
- ☐ 상품의 좋아요 수는 상품 상세/목록 조회에서 함께 제공된다
- ☐ 단위 테스트에서 좋아요 등록/취소 흐름을 검증했다

Order 도메인

- ☐ 주문은 여러 상품을 포함할 수 있으며, 각 상품의 수량을 명시한다
- ☐ 주문 시 상품의 재고 차감을 수행한다 (실제 결제 플로우 는 후주차에서 다룬다)
- ☐ 유저, 상품 부재, 재고 부족 등 예외 흐름을 고려해 설계되었다
- ☐ 단위 테스트에서 정상 주문 / 예외 주문 흐름을 모두 검증했다

도메인 서비스

- ☐ 도메인 간 협력 로직은 Domain Service에 위치시켰다
- ☐ 상품 상세 조회 시 Product + Brand 정보 조합은 도메인 서비스에서 처리했다
- ☐ 복합 유스케이스는 Application Layer에 존재하고, 도메인 로직은 위임되었다
- ☐ 도메인 서비스는 상태 없이, 도메인 객체의 협력 중심으로 설계되었다

소프트웨어 아키텍처 & 설계

- ☐ 전체 프로젝트의 구성은 아래 아키텍처를 기반으로 구성되었다
 - Application → **Domain** ← Infrastructure
- ☐ Application Layer는 도메인 객체를 조합해 흐름을 orchestration 했다
- ☐ 핵심 비즈니스 로직은 Entity, VO, Domain Service 에 위치한다
- ☐ Repository Interface는 Domain Layer 에 정의되고, 구현체는 Infra에 위치한다
- ☐ 패키지는 계층 + 도메인 기준으로 구성되었다 (`/domain/order` , `/application/like` 등)
- ☐ 테스트는 외부 의존성을 분리하고, Fake/Stub 등을 사용해 단위 테스트가 가능하게 구성되었다

Technical Writing Quest

피드백 & 라이팅 과제 - 테크노트 or 블로그 중 편한 방법으로 작성하고, 해당 링크를 제출해주시면 멘토님들이 이를 기반으로 피드백 및 RT 선정을 진행합니다.

1 GitHub Issues → New issue → **4개 포맷** 중 선택해서 작성

1.  **Design Doc** : 설계 의사결정 중심
2.  **Retrospective** : 과제 회고·트러블슈팅
3.  **Challenge Story** : 도전 → 해결 압축 서사
4.  **Benchmark Report** : A vs B 직접 측정

본문 골격이 템플릿에 따라 미리 채워져 있어요. 안 쓰는 섹션은 통째로 지우면 됩니다.

[포맷] 키워드 (N주차 · K팀 · 이름)

왜 도입했나요?이직·고과·면접에서 활용할 본인의 글쓰기 자산을 만드는 연습 트랙이에요. 잘 쓴 글은 멘토진이 RT 를 주거나 혹은 리뷰 시간에 소개해줄 거예요.**Issue 로 과제 시작하기**

- **Java:** <https://github.com/loopers-labs/loop-pack-be-l2-vol4-java/issues/new/choose>
- **Kotlin:** <https://github.com/loopers-labs/loop-pack-be-l2-vol4-kotlin/issues/new/choose>

가이드: 위 페이지의 "Tech Note 작성 가이드" 링크 참고

2 Blog 에 이번 주차 관련된 내용 을 기반으로 자유 작성

이번 주에 학습한 내용, 과제 진행을 되돌아보며
"내가 어떤 판단을 하고 왜 그렇게 구현했는지" 를 글로 정리해봅니다.
좋은 블로그 글은 내가 겪은 문제를, 타인도 공감할 수 있게 정리한 글입니다.

이 글은 단순 과제가 아니라, **향후 이직에 도움이 될 수 있는 포트폴리오** 가 될 수 있어요.

▼ Blog 작성 Tips!

✓ 작성 기준

항목	설명
형식	블로그
길이	제한 없음, 단 꼭 1줄 요약 (TL;DR) 을 포함해 주세요
포인트	"무엇을 했다" 보다 "왜 그렇게 판단했는가" 중심
예시 포함	코드 비교, 흐름도, 리팩토링 전후 예시 등 자유롭게
톤	실력은 보이지만, 자만하지 않고, 고민이 읽히는 글 예: "처음엔 mock으로 충분하다고 생각했지만, 나중에 fake로 교체하게 된 이유는..."

✨ 좋은 톤은 이런 느낌이에요

내가 겪은 실전적 고민을 다른 개발자도 공감할 수 있게 풀어내자

특징	예시
☹️ 내 언어로 설명한 개념	Stub과 Mock의 차이를 이번 주문 테스트에서 처음 실감했다
💬 판단 흐름이 드러나는 글	처음엔 도메인을 나누지 않았는데, 테스트가 어려워지며 분리했다
📌 정보 나열보다 인사이트 중심	테스트는 작성했지만, 구조는 만족스럽지 않다. 다음엔...

✗ 피해야 할 스타일

예시	이유
많이 부족했고, 반성합니다...	회고가 아니라 일기처럼 보입니다
Stub은 응답을 지정하고...	내 생각이 아닌 요약문처럼 보입니다
테스트가 진리다	너무 단정적이거나 오만해 보입니다

주제 추천

- 상품이 좋아요 수를 직접 관리해야 할까?
- 상품 상세에서 브랜드를 함께 제공하려면 누가 조합해야 할까?
- VO를 도입한 이유는 무엇이며, 어느 시점에서 유리하게 작용했는가?
- Order, Product, User 중 누가 어떤 책임을 갖는 것이 자연스러웠나?
- Repository Interface 를 Domain Layer에 두는 이유는?
- 처음엔 도메인에 두려 했지만, 결국 Application Layer로 옮긴 이유는?
- 테스트 가능한 구조를 만들기 위해 가장 먼저 고려한 건 무엇이었나?